

Osnove podatkovne znanosti



Fakultet informatike u Puli

Vježba 5

Funkcije, moduli i biblioteke

Predavanja: doc. dr. sc. Ivan Lorencin

Vježbe: Elvis Starić mag. inf.

U prethodnim vježbama smo podatke spremali u varijable i strukture, odabirali naredbe grananjima i ponavljali obradu petljama. Programi su pritom uglavnom bili jedan niz naredbi. Kako program raste, ista nam obrada može trebati na više mjesta: izračun prosjeka za nekoliko grupa, uređivanje više oznaka ili provjera različitih rezultata.

Funkcije omogućuju da takvu obradu jednom opišemo, damo joj ime i zatim je pozivamo s potrebnim podacima. U ovoj ćemo vježbi postupno povezati definiciju funkcije, podatke koje funkcija prima i rezultat koji vraća. Nakon toga vidjet ćemo kako funkcije organizirati u module i kako koristiti gotove funkcije iz Pythonove standardne biblioteke.

Primjere spremajte u mapu **Vježba 5** i pokrećite kao Python datoteke u VS Codeu. Potpuni primjeri sadrže definicije i pozive potrebne za samostalno pokretanje. Kada primjer koristi dvije datoteke, obje su posebno označene.

1 Zašto koristimo funkcije

1.1 Ponavljanje iste obrade

Za dvije grupe studenata želimo ispisati broj rezultata i prosjek bodova. Jedan mogući program zasebno računa podatke za svaku grupu:

```
bodovi_a = [60, 80, 100]
broj_a = len(bodovi_a)
prosjek_a = sum(bodovi_a) / broj_a
print("Grupa A")
print("Broj rezultata:", broj_a)
print(f"Prosjek: {prosjek_a:.2f}")

bodovi_b = [50, 70, 90, 90]
broj_b = len(bodovi_b)
prosjek_b = sum(bodovi_b) / broj_b
print("Grupa B")
print("Broj rezultata:", broj_b)
print(f"Prosjek: {prosjek_b:.2f}")
```

▼ Prikaži očekivani ispis

```
Grupa A
Broj rezultata: 3
Prosjek: 80.00
Grupa B
Broj rezultata: 4
Prosjek: 75.00
```

Oba dijela programa rade isti posao. Razlikuju se ulazni podaci i oznaka grupe, ali postupak izračuna i prikaza je jednak. Za treću grupu mogli bismo ponovno kopirati šest redaka i promijeniti nazive varijabli. Time bi se broj mjesta na kojima održavamo istu obradu dodatno povećao.

Prije nastavka, što bi trebali promijeniti kada bismo htjeli ispisivati prosjek na jednu decimalu. Što kada bismo trebali dodati provjeru ima li grupa uopće rezultate? Svaku promjenu trebalo bi dosljedno provesti u svim kopijama. Ako jednu zaboravimo, različite grupe obrađivale bi se prema različitim pravilima.

Petlja rješava ponavljanje nad nizom podataka. Funkcija rješava drugi dio problema: omogućuje da cijeli postupak imenujemo i koristimo gdje god nam treba. Funkciju možemo pozvati u petlji, nakon korisnikova unosa ili iz nekog drugog dijela programa. Ne moramo na svakom mjestu ponovno pisati njezine naredbe.

💡 Funkcija

Funkcija je imenovana programska cjelina koja pri pozivu izvršava određeni postupak. Može primiti ulazne podatke preko parametara i vratiti rezultat dijelu programa koji ju je pozvao.

1.2 Funkcije koje već koristimo

Pozivi `len(bodovi_a)`, `sum(bodovi_a)` i `print(...)` već su primjeri rada s funkcijama. Funkciji `len()` predajemo listu i dobivamo broj elemenata. Funkciji `sum()` predajemo brojeve u zbirci i dobivamo njihov zbroj. Ne pišemo svaki put petlju za brojanje ili zbrajanje jer je postupak već pripremljen.

Na sličan način možemo definirati vlastiti postupak. Razlika je u tome što sada mi određujemo naziv, ulazne podatke i naredbe koje se izvršavaju. Ugrađena funkcija `sum()` dio je Pythona, dok će naša funkcija postojati nakon izvršavanja njezine definicije.

Funkcija ne mora sadržavati složen izračun niti velik broj redaka. Smisao ima kada predstavlja razumljivu cjelinu: primjerice uređivanje oznake, računanje cijene ili provjeru rezultata. Samo premještanje proizvoljnih redaka pod nejasan naziv ne čini program preglednijim.

2 Definiranje i pozivanje funkcije

2.1 Od ponovljenih naredbi do jednog naziva

Pretpostavimo da pri početku svake obrade želimo prikazati isti naslov i napomenu. Bez funkcije ponavljali bismo sljedeće naredbe na svakom potrebnom mjestu:

```
print("OBRADA REZULTATA")
print("Bodovi se prikazuju u rasponu od 0 do 100.")
print("-----")
```

Taj blok možemo izdvojiti u funkciju `ispisi_zaglavlje`. Njezin naziv opisuje što se događa pri pozivu. Zapis počinje ključnom riječi `def`, iza koje slijede naziv, oble zagrade i dvotočka. Uvučene naredbe čine tijelo funkcije.

```
def ispisi_zaglavlje():
    print("OBRADA REZULTATA")
    print("Bodovi se prikazuju u rasponu od 0 do 100.")
    print("-----")

ispisi_zaglavlje()
print("Slijede rezultati prve grupe.")
```

▼ Prikaži očekivani ispis

```
OBRADA REZULTATA
Bodovi se prikazuju u rasponu od 0 do 100.
-----
Slijede rezultati prve grupe.
```

💡 Definicija i poziv funkcije

Definicija funkcije određuje naziv funkcije, njezine parametre i tijelo.

Poziv funkcije pokreće izvršavanje toga tijela s argumentima navedenima u pozivu.

Prazne zagrade u definiciji znače da funkcija nema parametara. U pozivu su zagrade i dalje potrebne:

`ispisi_zaglavlje()` pokreće funkciju. Sam naziv `ispisi_zaglavlje`, bez zagrada, ne pokreće njezino tijelo.

Uvlačenje ima istu važnost kao kod grananja i petlji. Sve tri naredbe `print()` pripadaju funkciji jer su uvučene. Poziv i završni ispis nisu uvučeni, pa pripadaju ostatku programa.

2.2 Kada se tijelo izvršava

Izvršavanje naredbe `def` i izvršavanje tijela funkcije događaju se u različitim trenucima. Kad Python dođe do naredbe `def`, stvara funkciju i povezuje je s nazivom. Naredbe u tijelu čekaju poziv. To možemo pratiti pomoću poruka prije i poslije definicije:

```
print("Početak programa")

def ispisi_poruku():
    print("Izvršava se tijelo funkcije")

print("Funkcija je definirana")
ispisi_poruku()
print("Nastavlja se ostatak programa")
```

▼ Prikaži očekivani ispis

```
Početak programa
Funkcija je definirana
Izvršava se tijelo funkcije
Nastavlja se ostatak programa
```

Redak koji ispisuje `Izvršava se tijelo funkcije` u datoteci nalazi se prije retka `Funkcija je definirana`, ali se izvršava kasnije. Python ne prolazi tijelom funkcije čim naiđe na njegov tekst. Tek pri pozivu prelazi na pripadajuće naredbe, a nakon njih nastavlja iza mjesta poziva.

Ako iz primjera uklonimo `ispisi_poruku()`, poruka `Izvršava se tijelo funkcije` neće se pojaviti. Funkcija je i dalje definirana, ali nije pozvana. Ako poziv napišemo dvaput, njezino će se tijelo izvršiti dvaput.

⚠ Funkcija mora biti definirana prije poziva

Prije poziva Python mora doći do naredbe `def` kako bi naziv funkcije postao poznat. Pritom se izvršava **naredba `def`**, ali se **tijelo funkcije ne izvršava**, ono čeka poziv. Ako poziv napišemo prije naredbe `def`, dobit ćemo `NameError`, čak i ako je definicija zapisana nekoliko redaka niže.

Nazivi funkcija slijede pravila naziva varijabli. Koristimo jasne nazive malim slovima i riječi odvajamo donjom crtom, primjerice `ispisi_zaglavljje`. Izbjegavamo nazive poput `print`, `sum` ili `list` (ili bilo koji drugi naziv ugrađene Python funkcije) jer bi se nakon takve definicije ti nazivi odnosili na naše funkcije, pa ugrađene Pythonove funkcije više ne možemo pozivati na uobičajen način.

📌 Pregled novih zapisa

Zapis	Namjena
<code>def</code>	Ključna riječ za definiranje funkcije

Zadatak

Na početku programa ispišite `Pokretanje programa`. Zatim definirajte funkciju `ispisi_kvadrata()` koja petljom ispisuje brojeve od `1` do `4` i kvadrat svakog broja, po jedan par u retku u obliku `broj -> kvadrat`. Nakon definicije ispišite `Funkcija je pripremljena`, pozovite funkciju jednom i na kraju ispišite `Kraj programa`.

▼ Prikaži moguće rješenje i ispis

```
print("Pokretanje programa")

def ispisi_kvadrata():
    for broj in range(1, 5):
        print(broj, "->", broj ** 2)

print("Funkcija je pripremljena")
ispisi_kvadrata()
print("Kraj programa")
```

```
Pokretanje programa
Funkcija je pripremljena
1 -> 1
2 -> 4
3 -> 9
4 -> 16
Kraj programa
```

3 Parametri i argumenti

3.1 Isti postupak s različitim podacima

Funkcija bez parametara uvijek ispisuje iste poruke. Ako bismo htjeli zasebno pozdraviti Anu i Ivu, mogli bismo definirati dvije funkcije, ali time bismo ponovno umnožili gotovo isti kod:

```
def pozdravi_anu():  
    print("Dobro došla, Ana!")  
  
def pozdravi_ivu():  
    print("Dobro došao, Ivo!")  
  
pozdravi_anu()  
pozdravi_ivu()
```

▼ Prikaži očekivani ispis

```
Dobro došla, Ana!  
Dobro došao, Ivo!
```

Za svakog novog studenta trebala bi nova funkcija. Želimo zadržati jedan postupak, a promjenjivi dio predati pri pozivu. To omogućuje parametar. U sljedećoj funkciji `ime` je naziv preko kojeg tijelo pristupa predanoj vrijednosti. Pozdrav smo oblikovali tako da ista poruka odgovara svakom imenu:

```
def pozdravi(ime):  
    print("Pozdrav,", ime)  
  
pozdravi("Ana")  
pozdravi("Ivo")
```

▼ Prikaži očekivani ispis

```
Pozdrav, Ana  
Pozdrav, Ivo
```

💡 Parametar i argument

Parametar je naziv naveden u definiciji funkcije preko kojeg funkcija pristupa ulaznom podatku. **Argument** je vrijednost ili izraz koji predajemo funkciji pri pozivu.

U definiciji `def pozdravi(ime):` riječ `ime` označava parametar. U pozivu `pozdravi("Ana")` tekst `"Ana"` argument je koji se povezuje s tim parametrom. Pri sljedećem pozivu parametar prima `"Ivo"`. Funkcija se ne prepisuje niti trajno pamti ime iz prethodnog poziva.

Argument ne mora biti doslovno zapisana vrijednost. Možemo predati varijablu ili rezultat izraza. Python najprije izračunava argument, a zatim ga predaje funkciji:

```
def pozdravi(ime):  
    print("Pozdrav,", ime)  
  
student = "Mia"  
pozdravi(student)  
pozdravi(" luka ".strip().title())  
print("Izvorno ime:", student)
```

▼ Prikaži očekivani ispis

```
Pozdrav, Mia  
Pozdrav, Luka  
Izvorno ime: Mia
```

Naziv varijable izvan funkcije ne mora biti jednak nazivu parametra. `student` i `ime` dva su različita naziva. Vrijednost `"Mia"` pri pozivu postaje dostupna u tijelu preko parametra `ime`. Funkciju zato možemo koristiti i s vrijednošću koja nije prethodno spremljena u varijablu.

3.2 Više parametara i njihov redoslijed

Za izračun vrijednosti narudžbe trebaju nam cijena jednog proizvoda i naručena količina. Jedan parametar nije dovoljan jer te dvije vrijednosti imaju različite uloge. Parametre u definiciji razdvajamo zarezima:

```
def ispisi_vrijednost(cijena, kolicina):  
    ukupno = cijena * kolicina  
    print(f"Vrijednost narudžbe: {ukupno:.2f} EUR")  
  
ispisi_vrijednost(12.50, 3)  
ispisi_vrijednost(8.00, 5)
```

▼ Prikaži očekivani ispis

```
Vrijednost narudžbe: 37.50 EUR  
Vrijednost narudžbe: 40.00 EUR
```

U prvom pozivu prvi argument `12.50` pripada parametru `cijena`, a drugi argument `3` parametru `kolicina`. Takve argumente nazivamo pozicijskima jer njihovu ulogu određuje položaj u pozivu.

Za ove obvezne parametre svaki poziv mora predati obje vrijednosti. Poziv `ispisi_vrijednost(12.50)` nema argument za količinu i završio bi s `TypeError`. Python ne pretpostavlja automatski da je količina `1`, takvo pravilo moramo posebno zadati.

Redoslijed je osobito važan kada zamjena argumenata mijenja značenje izračuna. U množenju se rezultat možda ne bi promijenio, ali kod razlike ili postotka pogrešan redoslijed može dati valjan, a besmislen odgovor.

3.3 Imenovani argumenti

Ako poziv sadrži više brojeva, iz samog zapisa može biti teško vidjeti što predstavljaju. Imenovani argument povezuje vrijednost s parametrom izričitim navođenjem njegova naziva:

```
def ispisi_postotak(ostvareno, moguće):  
    postotak = ostvareno / moguće * 100  
    print(f"Ostvareno: {postotak:.1f} %")
```

```
ispisi_postotak(18, 24)  
ispisi_postotak(moguće=24, ostvareno=18)
```

▼ Prikaži očekivani ispis

```
Ostvareno: 75.0 %  
Ostvareno: 75.0 %
```

Drugi poziv mijenja redoslijed pisanja argumenata, ali ne i njihovu povezanost s parametrima. `ostvareno` dobiva `18`, a `moguće` dobiva `24` zato što su nazivi navedeni. Oba poziva koriste istu funkciju i istu formulu. U ovom primjeru očekujemo da je mogući broj bodova veći od nule.

Naziv imenovanog argumenta mora odgovarati nazivu parametra u definiciji. Poziv s `maksimum=24` ne bi odgovarao parametru `moguće`. Možemo kombinirati pozicijske i imenovane argumente, primjerice `ispisi_postotak(18, moguće=24)`, pri čemu pozicijski argumenti dolaze prije imenovanih.

⚠ Jednom parametru predajemo jednu vrijednost

Poziv `ispisi_postotak(18, ostvareno=20, moguće=24)` pokušava parametru `ostvareno` predati i `18` i `20`. Python tada javlja `TypeError`. Nazivi argumenata ne stvaraju dodatne parametre, nego određuju koji postojeći parametar prima vrijednost.

i Zadatak

Napišite funkciju `ispisi_mjerenje` koja prima oznaku postaje i temperaturu te ispisuje rečenicu oblika `Postaja Pula: 22.5 °C`. Pozovite je za Pulu s temperaturom `22.5` i Pazin s temperaturom `18.0`. Jedan poziv napišite pozicijskim, a drugi imenovanim argumentima. U drugom pozivu temperaturu navedite prije oznake postaje.

▼ Prikaži moguće rješenje i ispis

```
def ispisi_mjerenje(postaja, temperatura):  
    print(f"Postaja {postaja}: {temperatura:.1f} °C")
```

```
ispisi_mjerenje("Pula", 22.5)  
ispisi_mjerenje(temperatura=18.0, postaja="Pazin")
```

Postaja Pula: 22.5 °C

Postaja Pazin: 18.0 °C

4 Vraćanje rezultata

4.1 Zašto ispis nije dovoljan

Dosadašnje funkcije uglavnom su ispisivale rezultat. To je korisno za prikaz, ali zamislimo da izračunatu vrijednost narudžbe trebamo povećati za trošak dostave ili spremiti u listu. Broj ispisan u terminalu nije automatski dostupan kao rezultat poziva.

Sljedeći primjer pokazuje što se događa kada pokušamo spremiti rezultat funkcije koja samo ispisuje:

```
def izracunaj_vrijednost(cijena, kolicina):  
    ukupno = cijena * kolicina  
    print(ukupno)  
  
rezultat = izracunaj_vrijednost(12.50, 3)  
print("Spremljena vrijednost:", rezultat)
```

▼ Prikaži očekivani ispis

```
37.5  
Spremljena vrijednost: None
```

Prvi broj ispisala je naredba unutar funkcije. Varijabla `rezultat` je `None` jer funkcija nema određenu vrijednost koju vraća. To je važna razlika: ispis je radnja koju funkcija obavlja, a povratna vrijednost podatak koji poziv daje ostatku programa.

Za predaju, odnosno vraćanje izračunatog broja koristimo `return`:

```
def izracunaj_vrijednost(cijena, kolicina):  
    ukupno = cijena * kolicina  
    return ukupno  
  
rezultat = izracunaj_vrijednost(12.50, 3)  
s_dostavom = rezultat + 4.00  
  
print(f"Proizvodi: {rezultat:.2f} EUR")  
print(f"S dostavom: {s_dostavom:.2f} EUR")
```

▼ Prikaži očekivani ispis

```
Proizvodi: 37.50 EUR  
S dostavom: 41.50 EUR
```

Povratna vrijednost je rezultat koji poziv funkcije predaje ostatku programa. Naredba `return` završava trenutni poziv funkcije i vraća vrijednost izraza zapisanog iza nje. Ako funkcija završi bez takve vrijednosti, vraća `None`.

`return` je ključna riječ, a ne funkcija: uobičajen zapis je `return ukupno`. Zagrade joj nisu potrebne. Kad program dođe do tog retka, broj `37.5` postaje rezultat cijelog izraza `izracunaj_vrijednost(12.50, 3)`. Dodjela ga zatim sprema u `rezultat`.

4.2 Korištenje povratne vrijednosti

Povratnu vrijednost možemo spremiti, odmah ispisati ili upotrijebiti unutar drugog izraza. Način korištenja određuje kod koji poziva funkciju. Funkcija koja računa vrijednost narudžbe zato može ostati jednaka bez obzira na to prikazujemo li račun ili računamo više narudžbi.

```
def izracunaj_vrijednost(cijena, kolicina):  
    return cijena * kolicina  
  
prva = izracunaj_vrijednost(5, 4)  
druga = izracunaj_vrijednost(3, 6)  
  
print("Prva:", prva)  
print("Druga:", druga)  
print("Ukupno:", prva + druga)  
print("Nova narudžba:", izracunaj_vrijednost(7, 2))
```

▼ Prikaži očekivani ispis

```
Prva: 20  
Druga: 18  
Ukupno: 38  
Nova narudžba: 14
```

U zadnjem retku najprije se izvršava unutarnji poziv `izracunaj_vrijednost(7, 2)`. Njegov rezultat `14` zatim se predaje funkciji `print()`. Takvo povezivanje već smo koristili u izrazima poput `print(len(lista))`, sada rezultat daje naša funkcija.

Samostalni poziv `izracunaj_vrijednost(7, 2)` u `.py` datoteci izvršio bi izračun, ali ne bi ništa ispisao. Ako povratnu vrijednost ne spremimo niti iskoristimo, ona se neće automatski prikazati. To nije kvar funkcije.

4.3 Završetak funkcije i None

Nakon izvršenog `return` ne nastavlja se ostatak tijela iste funkcije. Izvršavanje se vraća na mjesto poziva. Sljedeći primjer namjerno sadrži naredbu do koje program neće doći:

```
def udvostruci(broj):  
    return broj * 2  
    print("Ova poruka neće se ispisati.")  
  
print(udvostruci(6))
```

▼ Prikaži očekivani ispis

12

Naredba ispod `return` sintaksno je dopuštena, ali je u ovoj funkciji nedostižna. U stvarnom programu takav redak uklanjamo ili premještamo prije vraćanja rezultata, ovisno o namjeri.

Funkcija može imati i `return` bez izraza. Time odmah završava i vraća `None`. Jednaku povratnu vrijednost daje funkcija koja dođe do kraja tijela bez `return`. To je prihvatljivo za funkciju kojoj je svrha ispis, ali nije rezultat s kojim možemo računati kao s brojem.

⚠ Ispis i vraćanje vrijednosti različiti su postupci

`print(ukupno)` prikazuje vrijednost u terminalu. `return ukupno` predaje je ostatku programa. I sam `print()` vraća `None`, pa zapis `return print(ukupno)` ne vraća ispisani broj. Kada rezultat trebamo dalje obrađivati, vraćamo ga iz funkcije, a ispisujemo na mjestu poziva.

🔔 Pregled novih zapisa

Zapis

`return`

Namjena

Završava poziv funkcije i vraća zadanu vrijednost, odnosno `None` ako vrijednost nije navedena

📄 Zadatak

Dva videozapisa traju 2 minute i 45 sekundi te 3 minute i 20 sekundi. Za daljnju obradu treba odrediti njihovo ukupno trajanje u sekundama. Napišite funkciju `u_sekunde(minute, sekunde)` koja trajanje jednog videozapisa pretvara u sekunde i vraća dobiveni broj. Pozovite je za oba videozapisa, izvan funkcije zbrojite povratne vrijednosti i ispišite ukupan broj sekundi.

▼ Prikaži moguće rješenje i ispis

```
def u_sekunde(minute, sekunde):  
    video_sekunde = minute * 60 + sekunde  
    return video_sekunde  
  
prvi = u_sekunde(2, 45)  
drugi = u_sekunde(3, 20)  
ukupno = prvi + drugi  
print("Ukupno:", ukupno, "s")
```

Ukupno: 365 s

5 Grananja i petlje u funkcijama

5.1 Različiti rezultati za različite ulaze

Jedna formula dovoljna je kada sve podatke obrađujemo jednako. No za rezultat ispita želimo vratiti različitu poruku ovisno o bodovima. Bez funkcije ponavljali bismo isto grananje za svakog studenta kojem trebamo odrediti status.

Grananje možemo smjestiti u funkciju koja prima bodove i vraća tekst. Time pravilo o prolazu dobiva jedno mjesto u programu. U ovom primjeru ulazni bodovi trebaju biti broj u rasponu od 0 do 100. Najprije promatramo dva valjana rezultata:

```
def odredi_status(bodovi):  
    if bodovi >= 60:  
        return "Prolaz"  
    else:  
        return "Nije prolaz"  
  
print(odredi_status(72))  
print(odredi_status(45))
```

▼ Prikaži očekivani ispis

```
Prolaz  
Nije prolaz
```

U jednom pozivu izvršava se samo grana čiji je uvjet zadovoljen. Za 72 izvršava se prvi `return`, a za 45 drugi. Dva zapisa `return` ne znače da jedan poziv vraća dvije vrijednosti. Svaki izvršeni `return` završava taj poziv.

Kad funkcija treba vratiti rezultat za svaki valjani ulaz, sve moguće putanje trebaju ga odrediti. Ako uklonimo `else` i drugi `return`, poziv s 45 došao bi do kraja tijela i vratio `None`. Takva pogreška može ostati neprimijećena dok funkciju isprobavamo samo s prolaznim bodovima.

5.2 Provjera ulaza i raniji završetak

Nevaljan broj bodova i valjan rezultat ispod praga nisu isti slučaj. Broj 105 ne bi smio značiti prolaz samo zato što je veći od 60. Funkcija zato može najprije provjeriti raspon i završiti prije glavnog izračuna ako ulaz nije prihvatljiv.

```
def odredi_status(bodovi):
    if bodovi < 0 or bodovi > 100:
        return "Nevaljan broj bodova"

    elif bodovi >= 60:
        return "Prolaz"
    else:
        return "Nije prolaz"

print(odredi_status(105))
print(odredi_status(60))
print(odredi_status(59))
```

▼ Prikaži očekivani ispis

```
Nevaljan broj bodova
Prolaz
Nije prolaz
```

Za `105` funkcija završava na prvoj provjeri. Za `60` prva provjera nije zadovoljena pa program dolazi do sljedećeg uvjeta i vraća prolaz. Za `59` nijedan od prva dva uvjeta nije zadovoljen, pa se izvršava `else`.

Provjera raspona ne provjerava automatski tip. Ova funkcija očekuje broj. Poziv s tekstem `"72"` ne bi se mogao usporediti s brojem `0`. U zadacima zato treba razlikovati očekivani tip ulaza od dodatnih pravila o njegovoj vrijednosti.

5.3 Obrada cijele liste

Vratimo se prosjeku iz uvoda. Ako za svaku grupu zasebno pišemo zbroj i dijeljenje, isti postupak ponavljamo na više mjesta. Funkcija može primiti cijelu listu, izračunati rezultat i vratiti ga.

Što treba vratiti za praznu listu? Prosjek tada nije određen. Vrijednost `0` nije dobra zamjena jer bi značila stvarni prosjek nula. U ovom ćemo primjeru za praznu listu vratiti `None`, a kod koji poziva funkciju provjerit će rezultat.

```
def izracunaj_prosjek(vrijednosti):
    if len(vrijednosti) == 0:
        return None

    zbroj = 0
    for vrijednost in vrijednosti:
        zbroj += vrijednost

    return zbroj / len(vrijednosti)

print(izracunaj_prosjek([60, 80, 100]))
print(izracunaj_prosjek([]))
```

▼ Prikaži očekivani ispis

80.0
None

Akumulator `zbroj` postavlja se na `0` pri svakom pozivu. Petlja zatim obrađuje sve elemente predane liste. Završni `return` uvučen je kao `for`, pa se nalazi nakon petlje, unutar funkcije. Kada bi bio dodatno uvučen u tijelo petlje, funkcija bi završila već nakon prvog elementa.

Dosad smo povratne vrijednosti samo izravno ispisali: za listu s brojevima vidjeli smo `80.0`, a za praznu listu `None`. U izvještaju bismo za praznu listu radije ispisali objašnjenje, a za nepraznu prosjek na dvije decimale. Prikaz `:.2f` ne možemo primijeniti na `None`, pa nakon poziva najprije provjeravamo rezultat. Sljedeći kod koristi funkciju `izracunaj_prosjek()` definiranu iznad za listu `[0, 0, 0]`, koja ima valjan prosjek `0.0`, i za praznu listu, čiji prosjek ne postoji:

```
for mjerjenja in ([0, 0, 0], []):
    prosjek = izracunaj_prosjek(mjerjenja)
    if prosjek is None:
        print("Nema mjerjenja za izračun prosjeka.")
    else:
        print(f"Prosjek: {prosjek:.2f}")
```

▼ Prikaži očekivani ispis

Prosjek: 0.00
Nema mjerjenja za izračun prosjeka.

Zapis `prosjek is None` izričito provjerava je li vraćena vrijednost `None`. Za ovu namjenu koristimo `is None` ili `is not None`. Običan uvjet `if prosjek:` ne bi bio dovoljan: i broj `0` tumači se kao neistinit, iako je ovdje valjan prosjek. Za usporedbu bročanih ili tekstualnih vrijednosti i dalje koristimo `==`, a ne `is`.

5.4 Kada return pripada petlji

Ponekad nam upravo treba završetak tijekom petlje. Ako tražimo prvi negativan rezultat mjerenja, nakon njegova pronalaska više ne trebamo pregledavati ostatak liste.

```
def prvi_negativan(vrijednosti):
    for vrijednost in vrijednosti:
        if vrijednost < 0:
            return vrijednost

    return None

print(prvi_negativan([4, 2, -3, -8]))
print(prvi_negativan([4, 2, 0]))
```

▼ Prikaži očekivani ispis

-3
None

Prvi `return` namjerno je unutar petlje: završava funkciju čim se pronađe odgovarajući element. Završni `return None` nalazi se izvan petlje i izvršava se tek ako nijedan element nije negativan. Da smo ga stavili u `else` unutar petlje, funkcija bi odustala već nakon prvog nenegativnog broja.

⚠ return završava funkciju

`break` završava najbližu petlju, nakon čega se funkcija može nastaviti. `return` završava cijeli trenutačni poziv funkcije, čak i kada se nalazi unutar petlje. Njegov položaj mora odgovarati pitanju trebamo li obraditi sve elemente ili je dovoljan prvi pronađeni rezultat.

🔔 Pregled novog zapisa

Zapis

Namjena

`is None`

Provjerava je li vrijednost upravo `None`

📌 Zadatak

Napišite funkciju `broj_iznad_praga` koja prima listu temperatura i prag te vraća koliko je temperatura strogo veće od praga. Za podatke `[18, 25, 22, 27, 25]` i prag `25` rezultat treba biti `1`. Provjerite i praznu listu: u njoj nema temperature iznad praga pa rezultat treba biti `0`.

▼ Prikaži moguće rješenje i ispis

```
def broj_iznad_praga(temperature, prag):  
    broj = 0  
    for temperatura in temperature:  
        if temperatura > prag:  
            broj += 1  
    return broj  
  
print(broj_iznad_praga([18, 25, 22, 27, 25], 25))  
print(broj_iznad_praga([], 25))
```

1
0

Brojač se vraća nakon cijele petlje. Vrijednosti jednake pragu ne broje se jer je u zadatku traženo strogo veće.

6 Lokalne i globalne varijable

6.1 Gdje postoji varijabla iz funkcije

Pri računanju prosjeka funkcija koristi pomoćne varijable poput zbroja i broja elemenata. Ostatku programa često treba samo rezultat. Kada bi svaka pomoćna varijabla svake funkcije ostala zajednička cijelom programu, lako bismo slučajno promijenili podatak koji pripada drugom izračunu.

Python zato nazive nastale unutar funkcije u pravilu smješta u lokalni prostor toga poziva. Parametri su također lokalni nazivi. Njima se koristimo unutar funkcije, a nakon povratka rezultata ostatak programa ne dobiva automatski pristup tim nazivima. Varijable koje definiramo izvan funkcija, u glavnom dijelu datoteke, nazivamo globalnima u odnosu na funkcije te datoteke.

```
def površina_pravokutnika(sirina, visina):  
    površina = sirina * visina  
    return površina  
  
rezultat = površina_pravokutnika(4, 3)  
print("Rezultat:", rezultat)
```

▼ Prikaži očekivani ispis

Rezultat: 12

Unutar funkcije postoje nazivi `sirina`, `visina` i `površina`. Izvan nje broj `12` spremljen je pod nazivom `rezultat`. Nismo „izvezli” naziv `površina`: vratili smo njegovu vrijednost i dodijelili je drugoj varijabli.

Kad bismo na kraj primjera dodali `print(površina)`, Python bi javio `NameError` jer takav naziv nije definiran u vanjskom dijelu programa. Taj pokušaj nije potreban za korištenje rezultata, rezultat već imamo u varijabli `rezultat`.

💡 Doseg varijable

Doseg varijable označava dio programa u kojem varijablu možemo koristiti.

Lokalna varijabla nastaje unutar funkcije i dostupna je u pripadajućem pozivu funkcije.

Globalna varijabla definirana je izvan funkcija, u glavnom dijelu datoteke. Funkcija može pročitati njezinu vrijednost, ali obična dodjela istoimenom nazivu unutar funkcije stvara lokalnu varijablu.

6.2 Jednaki nazivi ne znače istu varijablu

U različitim dijelovima programa mogu se pojaviti jednaki nazivi. Funkcija može imati lokalnu varijablu `broj`, a glavni dio programa globalnu varijablu istoga naziva. Lokalna dodjela tada ne zamjenjuje globalnu vrijednost:

```
broj = 10

def prikazi_broj():
    broj = 3
    print("Unutar funkcije:", broj)

prikazi_broj()
print("Izvan funkcije:", broj)
```

▼ Prikaži očekivani ispis

```
Unutar funkcije: 3
Izvan funkcije: 10
```

Dodjela `broj = 3` pripada tijelu funkcije i stvara lokalni naziv. Zbog toga globalni `broj` ostaje `10`. Ovaj primjer pokazuje razliku doseg, ali u stvarnim programima smisleni nazivi često su jasniji od namjernog ponavljanja istog naziva za različite uloge.

Svaki novi poziv ponovno izvršava dodjele unutar tijela. Funkcija ne nastavlja automatski tamo gdje je prethodni poziv stao. Ako na početku postavimo brojač na nulu, svaki poziv započinje novim brojanjem.

6.3 Čitanje globalne varijable

Funkcija može pročitati globalnu varijablu ako unutar nje ne postoji istoimena lokalna varijabla. Za samo čitanje ne trebamo nikakvu posebnu naredbu. No rezultat funkcije tada može ovisiti o vrijednosti koja se ne vidi u pozivu:

```
naknada = 2

def cijena_s_naknadom(cijena):
    return cijena + naknada

print(cijena_s_naknadom(10))
naknada = 5
print(cijena_s_naknadom(10))
```

▼ Prikaži očekivani ispis

```
12
15
```

Poziv oba puta izgleda jednako, ali se globalna `naknada` promijenila. Za promjenjive ulazne podatke obično je jasnije dodati parametar, kao u ranijem primjeru `cijena_s_naknadom(cijena, naknada)`. Tada iz poziva vidimo koje vrijednosti utječu na rezultat.

6.4 Promjena globalne varijable

Čitanje globalne varijable i dodjeljivanje nove vrijednosti toj varijabli nisu ista stvar. Ako unutar funkcije želimo dodjelom promijeniti varijablu iz glavnog dijela programa, to moramo izričito označiti ključnom riječi `global`. Primjerice, jedan brojač može pratiti koliko je puta pozvana funkcija:

```
broj_obrađa = 0

def zabilježi_obrađu():
    global broj_obrađa
    broj_obrađa += 1

zabilježi_obrađu()
zabilježi_obrađu()
print("Broj obrađa:", broj_obrađa)
```

▼ Prikaži očekivani ispis

Broj obrađa: 2

Naredba `global broj_obrađa` govori Pythonu da dodjela u ovoj funkciji treba promijeniti globalnu varijablu, a ne stvoriti lokalnu istoga naziva. Bez te naredbe izraz `broj_obrađa += 1` ne bi radio: Python bi `broj_obrađa` smatrao lokalnom varijablom, ali bi pokušao pročitati njezinu vrijednost prije nego što joj je išta dodijeljeno. To bi završilo pogreškom `UnboundLocalError`.

⚠ Globalne varijable koristite promišljeno

`global` nije potreban da bi funkcija pročitala globalnu vrijednost, nego kada joj želi **dodijeliti novu vrijednost**. Iako je takav zapis dopušten, promjene globalnog stanja mogu otežati praćenje programa: rezultat ili učinak poziva ovisi o onome što se dogodilo ranije. Za većinu izračuna jasnije je predati podatke parametrima i vratiti rezultat pomoću `return`.

Pri radu s listama postoji dodatna razlika između promjene lokalnog naziva i promjene objekta na koji upućuje; nju ćemo posebno razraditi u poglavlju o strukturama.

🔔 Pregled novih zapisa

Zapis	Namjena
<code>global naziv</code>	Omogućuje da dodjela unutar funkcije promijeni globalnu varijablu toga naziva

Zadatak

U glavnom dijelu programa postavite `oznaka = "A"` i `broj_obrađa = 0`. Napišite funkciju `obradi_oznaku(oznaka)` koja predanoj oznaci dodaje nastavak `"-OK"` i vraća novu oznaku. Pri svakom pozivu funkcija treba povećati globalni `broj_obrađa`. Pozovite je najprije s varijablom `oznaka`, a zatim s tekстом `"B"`. Ispišite oba vraćena rezultata, izvornu vrijednost varijable `oznaka` i konačni `broj_obrađa`.

▼ Prikaži moguće rješenje i ispis

```
oznaka = "A"
broj_obrađa = 0

def obradi_oznaku(oznaka):
    global broj_obrađa
    broj_obrađa += 1
    uredena_oznaka = oznaka + "-OK"
    return uredena_oznaka

prva = obradi_oznaku(oznaka)
druga = obradi_oznaku("B")
print("Prva:", prva)
print("Druga:", druga)
print("Izvorna oznaka:", oznaka)
print("Broj obrađa:", broj_obrađa)
```

```
Prva: A-OK
Druga: B-OK
Izvorna oznaka: A
Broj obrađa: 2
```

7 Zadane vrijednosti parametara

7.1 Kada većina poziva koristi jednako pravilo

U dosadašnjim primjerima pri svakom smo pozivu funkciji predavali sve potrebne argumente. Zamislimo sada funkciju koja zaokružuje izmjerenu vrijednost. Većinu mjerenja želimo zaokružiti na dvije decimale, a samo ponekad na jednu ili tri. Kad bi broj decimala bio obavezan argument, u gotovo svakom pozivu morali bismo ponovno napisati `2`, iako se uobičajeno pravilo nije promijenilo.

Zato parametru `decimale` u definiciji možemo dodijeliti zadanu vrijednost `2`. Kada pri pozivu predamo samo mjerenje, funkcija će koristiti te dvije decimale. Ako želimo drukčiji broj decimala, navest ćemo ga kao drugi argument. Ta vrijednost vrijedi samo za taj poziv.

```
def zaokruzi_mjerenje(vrijednost, decimale=2):  
    return round(vrijednost, decimale)
```

```
print(zaokruzi_mjerenje(18.376))  
print(zaokruzi_mjerenje(18.376, 1))  
print(zaokruzi_mjerenje(18.376, decimale=3))
```

▼ Prikaži očekivani ispis

```
18.38  
18.4  
18.376
```

💡 Zadana vrijednost parametra

Zadana vrijednost parametra koristi se kada se u pozivu funkcije ne preda argument za taj parametar. Parametar koji ima definiranu zadanu vrijednost u pozivu možemo izostaviti.

Parametar `vrijednost` i dalje je obavezan. Parametar `decimale` ima zadanu vrijednost `2`, pa su svi prikazani pozivi valjani. Poziv s `decimale=3` ne mijenja definiciju funkcije, neki sljedeći poziv bez tog argumenta ponovno bi koristio `2`.

7.2 Obvezni i neobvezni parametri

U definicijama kakve ovdje pišemo Python zahtijeva da obvezni parametri stoje prije parametara sa zadanim vrijednostima. Obrnut redoslijed, primjerice `def primjer(a=1, b):`, izaziva `SyntaxError`. Zato funkciju za izračun dostave zapisujemo ovako:

```
def izracunaj_dostavu(tezina, cijena_po_kg=2, osnovna_naknada=3):  
    return tezina * cijena_po_kg + osnovna_naknada  
  
print(izracunaj_dostavu(4))  
print(izracunaj_dostavu(4, osnovna_naknada=0))  
print(izracunaj_dostavu(4))
```

▼ Prikaži očekivani ispis

```
11  
8  
11
```

Drugi poziv mijenja samo osnovnu naknadu. Cijena po kilogramu ostaje 2 jer je taj argument izostavljen. Imenovani argument ovdje omogućuje da navedemo baš vrijednost koju želimo promijeniti, bez ponovnog pisanja svih prethodnih zadanih vrijednosti.

Za parametre sa zadanim vrijednostima biramo najčešće pravilo koje ima jasno značenje. Vrijednost ne treba zadati samo kako bi svaki poziv mogao proći bez argumenata. Funkcija bez težine paketa ne bi imala podatak potreban za traženi izračun.

Zadatak

U popisu uzoraka zapisane su oznake " a7 ", " b2 " i " c9 ". Napišite funkciju `oznaci_uzorak` koja prima oznaku i neobvezni prefiks. Zadani prefiks neka bude "UZ-". Funkcija treba ukloniti razmake na rubovima oznake, pretvoriti njezina slova u velika i vratiti tekst sastavljen od prefiksa i uređene oznake. Pozovite funkciju za prvu oznaku bez navođenja prefiksa, za drugu s prefiksom "LAB-", a za treću ponovno bez navođenja prefiksa. Ispišite sva tri rezultata.

▼ Prikaži moguće rješenje i ispis

```
def oznaci_uzorak(oznaka, prefiks="UZ-"):  
    return prefiks + oznaka.strip().upper()  
  
print(oznaci_uzorak(" a7 "))  
print(oznaci_uzorak(" b2 ", prefiks="LAB-"))  
print(oznaci_uzorak(" c9 "))
```

```
UZ-A7  
LAB-B2  
UZ-C9
```

8 Funkcije i složeni tipovi podataka

8.1 Promjena predane liste

Kod broja smo vidjeli da dodjela novoj vrijednosti parametra ne mijenja vanjsku varijablu. Kod liste moramo razlikovati dvije radnje: **promjenu sadržaja postojeće liste** i **dodjelu druge liste lokalnom parametru**. Obje se mogu dogoditi unutar funkcije, ali nemaju isti učinak izvan nje.

Kada funkciji predamo listu, Python je ne kopira automatski. Varijabla kojom smo je predali i parametar funkcije tada upućuju na istu listu. U sljedećem primjeru `bodovi` je naziv izvan funkcije, a `rezultati` naziv koji funkcija koristi za tu istu listu.

Ako program u funkciji doda element toj listi, promjena će biti vidljiva i nakon završetka funkcije:

```
def dodaj_rezultat(rezultati, novi_rezultat):
    rezultati.append(novi_rezultat)
```

```
bodovi = [70, 80]
dodaj_rezultat(bodovi, 90)
print(bodovi)
```

▼ Prikaži očekivani ispis

```
[70, 80, 90]
```

Pratimo što se događa pri pozivu:

1. Naredba `bodovi = [70, 80]` stvara listu. Varijabla `bodovi` upućuje na nju.
2. Poziv `dodaj_rezultat(bodovi, 90)` predaje funkciji tu postojeću listu. Parametar `rezultati` postaje još jedan naziv za istu listu, a ne naziv za njezinu kopiju.
3. Naredba `rezultati.append(90)` dodaje element u **tu listu**. Zato se njezin sadržaj mijenja iz `[70, 80]` u `[70, 80, 90]`.
4. Po završetku poziva lokalni naziv `rezultati` više nam nije dostupan, ali lista nije nestala: varijabla `bodovi` i dalje upućuje na nju pa vidimo dodani element.

Funkcija ne vraća novu listu pomoću `return`. Njezin je učinak upravo promjena liste koju je primila.

Takva izmjena može biti namjerna. Naziv `dodaj_rezultat` upravo najavljuje dodavanje. Važno je da osoba koja koristi funkciju zna mijenja li ona ulaz ili stvara novi rezultat.

8.2 Nova lokalna vrijednost nije promjena izvornog objekta

Sada polazimo od iste liste `[70, 80]` i ponovno je predajemo funkciji. Na početku poziva `bodovi` i `rezultati` opet upućuju na istu listu. Razlika je u naredbi unutar funkcije: ovaj put ne pozivamo metodu koja mijenja listu, nego parametru dodjeljujemo novu vrijednost.

```
def postavi_praznu_listu(rezultati):
    rezultati = []
    print("Unutar funkcije:", rezultati)
```

```
bodovi = [70, 80]
postavi_praznu_listu(bodovi)
print("Izvan funkcije:", bodovi)
```

▼ Prikaži očekivani ispis

```
Unutar funkcije: []
Izvan funkcije: [70, 80]
```

Usporedimo korake s prethodnim primjerom:

1. Pri pozivu `postavi_praznu_listu(bodovi)` parametar `rezultati` najprije upućuje na istu listu kao `bodovi`.
2. Naredba `rezultati = []` stvara **novu praznu listu** i povezuje lokalni naziv `rezultati` s njom. Ne prazni listu `[70, 80]` koju smo predali funkciji.
3. Ispis unutar funkcije prikazuje novu listu `[]`, jer naziv `rezultati` sada upućuje na nju.
4. Izvan funkcije `bodovi` i dalje upućuje na izvornu listu `[70, 80]`, pa završni ispis ostaje nepromijenjen.

Da smo umjesto dodjele `rezultati = []` pozvali `rezultati.clear()`, ispraznili bismo **postojeću** listu. Tada bi i ispis varijable `bodovi` izvan funkcije prikazao `[]`.

Ovu razliku ne određuje samo mjesto naredbe unutar funkcije. Određuje je operacija: mijenjamo li objekt ili preusmjeravamo lokalni naziv na drugi objekt. Isti princip vrijedi za promjenjive rječnike i skupove.

8.3 Vraćanje nove liste

Ako pri obradi želimo sačuvati ulazne podatke, funkcija može napraviti novu listu. Primjerice, korekcija mjerenja treba vratiti nove temperature, dok izvorna mjerenja ostaju dostupna za usporedbu:

```
def korigiraj_temperature(temperature, pomak):
    korigirane = []
    for temperatura in temperature:
        korigirane.append(temperatura + pomak)
    return korigirane
```

```
izvorne = [18, 20, 22]
nove = korigiraj_temperature(izvorne, 1)

print("Izvorne:", izvorne)
print("Korigirane:", nove)
```

▼ Prikaži očekivani ispis

Izvorne: [18, 20, 22]

Korigirane: [19, 21, 23]

Funkcija samo čita elemente ulazne liste. Nove vrijednosti dodaje u zasebnu listu nastalu unutar poziva i vraća je na kraju. Prazna ulazna lista prirodno daje praznu izlaznu listu jer petlja nema iteracija.

Za ovu novu organizaciju ne trebamo novi Pythonov tip niti novu metodu. Povezujemo poznato stvaranje liste, petlju, dodavanje i vraćanje rezultata. Zato je važno čitati ulogu svake varijable: `temperature` je ulaz, a `korigirane` novi rezultat.

8.4 Više povezanih rezultata

Za jednu listu mjerenja mogu nam trebati najmanja i najveća vrijednost. Mogli bismo napisati dvije funkcije, ali možemo vratiti i jednu n-torku s oba podatka.

```
def granice_mjerenja(vrijednosti):  
    if len(vrijednosti) == 0:  
        return None  
    return min(vrijednosti), max(vrijednosti)
```

```
granice = granice_mjerenja([18, 25, 21, 16])
```

```
if granice is not None:  
    najmanje, najvece = granice  
    print("Najmanje:", najmanje)  
    print("Najveće:", najvece)
```

▼ Prikaži očekivani ispis

Najmanje: 16

Najveće: 25

Zapis `return min(vrijednosti), max(vrijednosti)` vraća jednu vrijednost: n-torku koja sadrži dva broja. Izraz „funkcija vraća više rezultata” ovdje znači da su rezultati zapakirani u jednu strukturu. Raspakiravanje daje nazive svakom dijelu (Vježba 4).

Provjera `is not None` prethodi raspakiravanju jer prazan ulaz nema dvije granice koje bismo raspakirali. Za više imenovanih svojstava rezultat može biti rječnik:

```
def sazetak_mjerenja(vrijednosti):
    if len(vrijednosti) == 0:
        return {"broj": 0, "prosjek": None}

    return {
        "broj": len(vrijednosti),
        "prosjek": sum(vrijednosti) / len(vrijednosti)
    }

sazetak = sazetak_mjerenja([18, 20, 22])
print("Broj:", sazetak["broj"])
print("Prosjek:", sazetak["prosjek"])
```

▼ Prikaži očekivani ispis

Broj: 3
Prosjek: 20.0

Rezultat ima ključeve istih naziva i za praznu i za nepraznu listu, a vrijednost prosjeka pokazuje postoji li izračun. Funkcija se ne mora baviti izgledom izvještaja: glavni dio programa odabire što će iz vraćenog rječnika prikazati.

8.5 Pozivanje funkcije iz druge funkcije

Ponekad dio obrade već imamo opisan u vlastitoj funkciji. Ako znamo urediti jednu oznaku, isti postupak ne trebamo ponovno pisati u funkciji koja uređuje cijelu listu. Funkcija može pozvati drugu funkciju i iskoristiti njezin rezultat, jednako kao što već poziva ugrađene funkcije poput `len()`.

```
def uredi_jednu_oznaku(oznaka):
    return oznaka.strip().upper()

def uredi_popis(oznake):
    uredene = []
    for oznaka in oznake:
        uredene.append(uredi_jednu_oznaku(oznaka))
    return uredene

print(uredi_popis([" a1 ", " b2 "]))
```

▼ Prikaži očekivani ispis

['A1', 'B2']

Pri pozivu `uredi_popis()` počinje petlja. U svakoj iteraciji poziva se `uredi_jednu_oznaku()`, čeka se njezin rezultat i taj se tekst dodaje u novu listu. Njezin `return` završava samo poziv koji uređuje jednu oznaku. Funkcija s petljom nakon toga nastavlja rad. Tek nakon svih oznaka vanjska funkcija vraća cijelu listu.

Takva podjela korisna je ako uređivanje pojedinačne oznake trebamo i na drugim mjestima. Promjenu pravila, primjerice dodatnu zamjenu znakova, tada možemo napraviti u jednoj funkciji. Ne mora svaki pojedinačni izraz dobiti vlastitu funkciju: podjela treba pratiti razumljive postupke u programu.

Zadatak

Oznaku smatramo potpunom ako nakon uklanjanja razmaka na početku i kraju ima točno tri znaka. Napišite funkciju `oznaka_je_potpuna(oznaka)` koja to provjerava i vraća logičku vrijednost. Zatim napišite funkciju `prebroji_potpune(oznake)` koja prolazi listom, za svaku oznaku poziva prvu funkciju i vraća broj potpunih oznaka. Za listu `[" A01 ", "B2", " C03", "D004", " "]` ispišite taj broj i izvornu listu kako biste provjerili da je ostala nepromijenjena.

▼ Prikaži moguće rješenje i ispis

```
def oznaka_je_potpuna(oznaka):
    return len(oznaka.strip()) == 3

def prebroji_potpune(oznake):
    broj = 0
    for oznaka in oznake:
        if oznaka_je_potpuna(oznaka):
            broj += 1
    return broj

izvorne = [" A01 ", "B2", " C03", "D004", " "]
print("Broj potpunih oznaka:", prebroji_potpune(izvorne))
print("Izvorne:", izvorne)
```

```
Broj potpunih oznaka: 2
Izvorne: [' A01 ', 'B2', ' C03', 'D004', ' ']
```

Prva funkcija provjerava jednu oznaku. Druga koristi njezin logički rezultat pri brojanju i ne mijenja ulaznu listu; `strip()` vraća novi tekst.

9 Moduli i biblioteke

9.1 Kada već postoji gotova funkcija

Prosjeck smo već računali izrazom `sum(vrijednosti) / len(vrijednosti)`. Za neke druge statistike postupak ima više koraka. Medijan, primjerice, zahtijeva poredane podatke i različit izračun ovisno o tome imamo li paran ili neparan broj vrijednosti.

Za nepraznu listu mogli bismo sami napisati sljedeći postupak:

```
mjerjenja = [19, 30, 21, 20]
poredana = sorted(mjerjenja)
broj = len(poredana)
sredina = broj // 2

if broj % 2 == 1:
    medijan = poredana[sredina]
else:
    medijan = (poredana[sredina - 1] + poredana[sredina]) / 2

print("Medijan:", medijan)
```

▼ Prikaži očekivani ispis

Medijan: 20.5

Nakon sortiranja dobivamo `[19, 20, 21, 30]`. Budući da su četiri vrijednosti, medijan je prosjek srednjih dviju, odnosno `20.5`. Važno je razumjeti taj postupak kako bismo znali što rezultat predstavlja.

No kada nam medijan treba u stvarnoj analizi, ne moramo svaki put ponovno pisati vlastitu izvedbu. Python sadrži modul `statistics` s već pripremljenim statističkim funkcijama. Da bismo ih koristili, modul uvozimo naredbom `import`:

```
import statistics

mjerjenja = [19, 30, 21, 20]
print("Medijan:", statistics.median(mjerjenja))
```

▼ Prikaži očekivani ispis

Medijan: 20.5

Postupak izračuna sada je dostupan kroz jasan naziv. I dalje trebamo znati kakve podatke funkcija očekuje i što vraća. Gotova funkcija ne odlučuje umjesto nas je li medijan primjeren odgovor na pitanje koje istražujemo.

💡 Modul, paket i biblioteka

Modul je organizacijska cjelina Pythona s vlastitim prostorom naziva. Često je zapisan kao jedna `.py` datoteka koja sadrži povezane funkcije, varijable i druge definicije.

Paket organizira povezane module, a može sadržavati i druge pakete.

Biblioteka je naziv za zbirku gotovih alata namijenjenih određenim zadacima. Jedna biblioteka može obuhvaćati više paketa i modula.

Ti pojmovi opisuju povezane, ali različite stvari. Modul `statistics` jedan je dio Pythonove standardne biblioteke. NumPy je biblioteka za numerički rad organizirana u paket s više modula. U oba slučaja funkcije koristimo preko naziva koje učinimo dostupnima uvozom.

9.2 Uvoz modula i pristup njegovim funkcijama

Bez prethodnog uvoza naziv `statistics` nije dostupan u našoj datoteci. Naredba `import statistics` učitava modul ako već nije učitani i povezuje ga s tim nazivom u trenutačnom programu. Zatim točkom pristupamo njegovim funkcijama, primjerice `statistics.mean()`.

```
import statistics

temperature = [18, 20, 22, 28]
prosjeak = statistics.mean(temperature)
medijan = statistics.median(temperature)

print("Prosjeak:", prosjeak)
print("Medijan:", medijan)
print("Izvorni podaci:", temperature)
```

▼ Prikaži očekivani ispis

```
Prosjeak: 22
Medijan: 21.0
Izvorni podaci: [18, 20, 22, 28]
```

`statistics.mean()` vraća aritmetičku sredinu, a `statistics.median()` medijan. Obje ovdje dobivaju istu listu, ali odgovaraju na različita pitanja. Izvorna lista ostaje nepromijenjena.

Zapis `statistics.mean(temperature)` sastoji se od naziva modula, točke, naziva funkcije i argumenata u zagradama. Točka ovdje označava pristup funkciji unutar modula. Kod `temperature.append(...)` točkom smo pristupali metodi liste. Zapis je sličan, ali `mean()` je funkcija modula, dok je `append()` metoda vezana uz listu.

⚠ Gotove funkcije imaju uvjete korištenja

`statistics.mean([])` i `statistics.median([])` ne mogu izračunati rezultat prazne liste i javljaju `StatisticsError`. Naša funkcija za prosjek ranije je u tom slučaju vraćala `None` jer smo sami tako odredili njezino

ponašanje. Jednaka namjena ne jamči jednako postupanje sa svim ulazima.

9.3 Modul math

Za geometrijski izračun mogu nam trebati kvadratni korijen ili broj π . Mogli bismo ručno zapisivati približnu vrijednost π i vlastite izraze za računanje, ali modul `math` pruža standardne matematičke funkcije i konstante.

```
import math

polumjer = 3
povrsina = math.pi * polumjer ** 2
dijagonala = math.sqrt(3 ** 2 + 4 ** 2)

print(f"Površina kruga: {povrsina:.2f}")
print("Dijagonala pravokutnika:", dijagonala)
```

▼ Prikaži očekivani ispis

```
Površina kruga: 28.27
Dijagonala pravokutnika: 5.0
```

`math.sqrt()` funkcija je koju pozivamo s argumentom. `math.pi` već je brojčana vrijednost, pa iza nje ne pišemo zagrada. Poziv `math.pi()` pokušao bi broj koristiti kao funkciju i izazvao `TypeError`.

Kvadratni korijen ovdje računamo za nenegativan broj. `math.sqrt(-1)` javlja `ValueError` jer ta funkcija ne vraća kompleksni korijen. U ovoj vježbi radimo s realnim brojevima i ulazima iz dopuštenog područja.

9.4 Različiti oblici uvoza

Ponekad želimo kraći naziv modula ili izravno koristiti jednu njegovu funkciju. Python nudi nekoliko oblika uvoza. Ne mijenjaju izračun funkcije, nego nazive kojima joj pristupamo u programu.

Prvi oblik zadržava naziv modula u svakom pozivu:

```
import math

print(math.sqrt(81))
```

▼ Prikaži očekivani ispis

```
9.0
```

Drugi oblik uvozi odabrani naziv iz modula. Funkcija `sqrt` tada je izravno dostupna:

```
from math import sqrt

print(sqrt(81))
```

▼ Prikaži očekivani ispis

9.0

Treći oblik daje modulu drugi lokalni naziv, odnosno alias:

```
import statistics as stat

print(stat.median([2, 8, 4]))
```

▼ Prikaži očekivani ispis

4

Alias `stat` ne mijenja naziv modula `statistics`. Vrijedi u datoteci u kojoj smo napisali taj uvoz. Nakon ovog oblika uvoza funkciju pozivamo preko `stat`; sam naziv `statistics` nije uveden tom naredbom.

Kod `from math import sqrt` dostupan je naziv `sqrt`, ali ta naredba ne uvodi i naziv `math`. Zato oblik poziva treba odgovarati obliku uvoza. Ne miješamo `sqrt(...)` i `math.sqrt(...)` bez odgovarajućeg uvoza.

Za početne primjere često je pregledan oblik `import modul` jer iz poziva vidimo odakle funkcija dolazi. U praksi ćemo susresti i ustaljene aliase poput `np` za NumPy te `plt` za modul `matplotlib.pyplot`. To su dogovoreni nazivi, a ne posebne Pythonove ključne riječi.

⚠ Izbjegavajte uvoz svih naziva

Zapis `from math import *` uvodi puno naziva odjednom. Time postaje teže vidjeti odakle neka funkcija dolazi i lakše je zakloniti vlastiti naziv. U primjerima uvozimo cijeli modul pod jednim nazivom ili izričito navedemo funkciju koja nam treba.

9.5 Čitanje dokumentacije funkcije

Nije potrebno pamtit sve funkcije nekog modula. Ako najprije želimo vidjeti koje nazive modul nudi, možemo upotrijebiti ugrađenu funkciju `dir()`:

```
import math

print(dir(math))
```

Dobit ćemo listu naziva iz modula `math`. Među njima su funkcije poput `sqrt`, ali i vrijednosti poput `pi` te posebni nazivi koji počinju dvjema donjim crtama. `dir()` zato ne izdvaja samo funkcije i ne objašnjava kako se pojedini naziv koristi. Popis se može razlikovati među verzijama Pythona.

Kada odaberemo funkciju koja nas zanima, možemo zatražiti pomoć samo za nju. Važno je navesti **naziv konkretne funkcije**: `help()` bez argumenta otvara interaktivnu pomoć, a `help(math)` prikazuje mnogo informacija o cijelom modulu. Za početak je preglednije pogledati jednu funkciju.

⚠ Kako izaći iz prikaza pomoći

Pri pokretanju `help(math.sqrt)` u terminalu se može otvoriti zaseban prikaz teksta s oznakom `END` na dnu. To nije pogreška. Pritisnite tipku `q` kako biste zatvorili prikaz pomoći i vratili se u terminal.

```
import math

help(math.sqrt)
```

▼ Prikaži primjer ispisa i objašnjenje

Help on built-in function sqrt in module math:

```
sqrt(x, /)
    Return the square root of x.
```

Vidimo da funkcija pripada modulu `math`, prima argument `x` i vraća njegov kvadratni korijen. Znak `/` označava da `x` predajemo položajem: pišemo `math.sqrt(81)`, a ne `math.sqrt(x=81)`. Točan prikaz pomoći može se razlikovati prema verziji Pythona.

Ovaj primjer ispisuje pomoć za funkciju, a ne kvadratni korijen. Funkciji `help()` predajemo naziv `math.sqrt` bez poziva. Da napišemo `help(math.sqrt(81))`, prvo bismo izračunali `9.0` i zatim tražili pomoć za dobivenu vrijednost.

Ako kratki opis nije dovoljan, dodatne primjere i ograničenja možemo potražiti u dokumentaciji modula.

Opis funkcija može se otvoriti i kroz prijedloge uređivača. Službena dokumentacija nudi dodatne primjere i objašnjenja: modul `math` i modul `statistics`.

🔔 Pregled novih naredbi i funkcija

Zapis	Namjena
<code>import statistics</code>	Uvozi modul pod njegovim nazivom
<code>from math import sqrt</code>	Uvozi odabranu funkciju kao izravno dostupan naziv
<code>import statistics as stat</code>	Uvozi modul pod aliasom
<code>statistics.mean(vrijednosti)</code>	Vraća aritmetičku sredinu nepraznih brojčanih podataka

Zapis

Namjena

`statistics.median(vrijednosti)`

Vraća medijan nepraznih brojčanih podataka

`math.sqrt(broj)`

Vraća kvadratni korijen nenegativnog broja

`math.pi`

Daje brojčanu vrijednost konstante π

`dir(math)`

Vraća popis naziva dostupnih u modulu, ne samo funkcija

`help(math.sqrt)`

Prikazuje pomoć za funkciju

Zadatak

Promjeri četiriju kružnih uzoraka iznose `[2, 4, 4, 10]` centimetara. Prosječni promjer odredite funkcijom `statistics.mean()`, a medijalni funkcijom `statistics.median()`. Zatim pomoću vrijednosti `math.pi` izračunajte površinu kruga kojem je promjer jednak prosječnom promjeru uzoraka i površinu kruga kojem je promjer jednak medijalnom promjeru. Ispišite oba promjera i obje površine na dvije decimale.

Površina kruga - $P = \pi \cdot r^2$

▼ Prikaži moguće rješenje i ispis

```
import math
import statistics

promjeri = [2, 4, 4, 10]
prosjecni_promjer = statistics.mean(promjeri)
medijalni_promjer = statistics.median(promjeri)

povrsina_prema_prosjeku = math.pi * (prosjecni_promjer / 2) ** 2
povrsina_prema_medijanu = math.pi * (medijalni_promjer / 2) ** 2

print(f"Prosječni promjer: {prosjecni_promjer:.2f} cm")
print(f"Medijalni promjer: {medijalni_promjer:.2f} cm")
print(f"Površina prema prosjeku: {povrsina_prema_prosjeku:.2f} cm²")
print(f"Površina prema medijanu: {povrsina_prema_medijanu:.2f} cm²")
```

Prosječni promjer: 5.00 cm

Medijalni promjer: 4.00 cm

Površina prema prosjeku: 19.63 cm²

Površina prema medijanu: 12.57 cm²

Uzorak promjera 10 cm podiže prosjek na 5 cm, dok je medijan 4 cm. Medijalni promjer ovdje bolje opisuje većinu uzoraka jer se tri od četiri promjera nalaze na 4 cm ili ispod te vrijednosti. Površine su izračunate iz polumjera, pa prije uporabe izraza πr^2 svaki promjer dijelimo s 2.

10 Vlastiti modul i organizacija programa

10.1 Ista funkcija u više datoteka

Pretpostavimo da funkciju za uređivanje oznake koristimo i pri obradi studenata i pri obradi proizvoda. Ako je kopiramo u dvije datoteke, promjenu pravila ponovno moramo provesti na dva mjesta. Funkciju možemo izdvojiti u vlastiti modul i uvesti je u svaki program kojem treba.

Za ovaj primjer napravite dvije datoteke u istoj mapi. Modul nazovite `obrada.py`, a program koji ga koristi `primjer_modula.py`. Pripremljene datoteke raspoređene su ovako:

Vježba 5/

```
└─ primjeri/
   └─ obrada.py
   └─ primjer_modula.py
```

Gotove datoteke za provjeru dostupne su i u mapi `primjeri` uz skriptu: `obrada.py` i `primjer_modula.py`. Ako ih preuzimate, spremite obje u istu mapu.

Datoteka `obrada.py` sadrži definiciju funkcije:

```
def uredi_oznaku(oznaka):
    return oznaka.strip().upper()
```

Datoteka `primjer_modula.py` uvozi modul i poziva njegovu funkciju:

```
import obrada

print(obrada.uredi_oznaku(" s-017 "))
print(obrada.uredi_oznaku(" p-204 "))
```

▼ Prikaži očekivani ispis pri pokretanju primjer_modula.py

```
S-017
P-204
```

U naredbi `import obrada` pišemo naziv modula bez nastavka `.py`. Točka u `obrada.uredi_oznaku(...)` znači pristup funkciji iz tog modula. Ako pokrenemo samo `obrada.py`, ništa se neće ispisati jer datoteka samo definira funkciju, njezino se tijelo izvršava tek pri pozivu iz `primjer_modula.py`.

⚠ Naziv datoteke može zakloniti postojeći modul

Vlastite datoteke nemojte nazivati `math.py`, `statistics.py`, `numpy.py` ili `matplotlib.py` kada želite koristiti te module. Python može pronaći vašu datoteku umjesto očekivanog modula, pa tražene funkcije neće biti dostupne. Nazivi poput `obrada.py` i `analiza_temperatura.py` jasnije opisuju sadržaj.

i Zadatak

U datoteci `pretvorbe.py` definirajte funkciju `minute_u_sate` koja prima broj minuta i vraća broj sati. U zasebnoj datoteci `trajanja.py` pomoću te funkcije izračunajte trajanje od `90` i `135` minuta u satima.

▼ Prikaži moguće rješenje i ispis

Datoteka `pretvorbe.py`:

```
def minute_u_sate(minute):  
    return minute / 60
```

Datoteka `trajanja.py`:

```
import pretvorbe  
  
print(pretvorbe.minute_u_sate(90))  
print(pretvorbe.minute_u_sate(135))
```

Ispis pri pokretanju `trajanja.py`:

```
1.5  
2.25
```

11 Provjera funkcija i česte pogreške

Poruka o pogrešci korisna je ako je povežemo s mjestom i oblikom poziva. `NameError` kod naziva funkcije može značiti da je nismo definirali ili uvezli. `TypeError` zbog nedostajućeg argumenta znači da poziv ne odgovara parametrima definicije.

Pojava	Mogući uzrok	Što provjeriti
Funkcija je zapisana, ali nema očekivanog ispisa	Funkcija nije pozvana ili samo vraća rezultat	Postoji li poziv i koristi li se njegov rezultat
<code>NameError</code> pri pozivu	Naziv još nije dostupan ili je pogrešno napisan	Definiciju, uvoz i naziv u pozivu
<code>TypeError</code> zbog argumenata	Nedostaje argument ili je isti predan dvaput	Parametre definicije i argumente poziva
Rezultat je <code>None</code>	Funkcija samo ispisuje ili neka grana nema povratnu vrijednost	Sve putanje do završetka funkcije
Obraden je samo prvi element liste	<code>return</code> je prerano unutar petlje	Razinu uvlačenja završnog rezultata
Izvorna lista neočekivano je promijenjena	Funkcija mijenja predani objekt	Pozive metoda poput <code>append()</code> i <code>sort()</code>
<code>ModuleNotFoundError</code>	Python nije pronašao zadani modul	Naziv u naredbi <code>import</code> , lokaciju vlastite datoteke i odabrani interpreter
Modul nema očekivanu funkciju	Uvezen je drugi modul ili je naziv pogrešan	Naziv vlastite datoteke i naziv tražene funkcije

Kod pogreške pri uvozu nije dovoljno zaključiti da „Python ne radi”. Vlastiti modul može biti spremljen u pogrešnoj mapi ili datoteka može imati pogrešan naziv. Provjeravamo konkretan naziv naveden u poruci i način na koji pokrećemo program.

Za pitanja o primjerima možete se javiti putem Google Chata ili e-maila. Priložite kod koji pokrećete i cijelu poruku pogreške, a kod problema s modulima i nazive te raspored datoteka.

12 Pregled naredbi, funkcija i metoda

Funkciju možemo razumjeti kroz tri dijela: podatke koje prima, postupak koji izvodi i rezultat koji vraća. Parametri povezuju poziv s ulaznim podacima, lokalne varijable služe obradi, a `return` određuje rezultat. Ako funkcija mijenja predanu strukturu ili ispisuje poruku, taj je učinak također dio njezina ponašanja.

Moduli proširuju isti princip na organizaciju datoteka. Vlastite funkcije ili gotove alate uvozimo i pozivamo prema njihovim nazivima. Uvoz ne mijenja osnovna pravila poziva: argumenti moraju odgovarati funkciji, a rezultat se koristi prema njegovu tipu i značenju.

Sljedeće tablice služe kao podsjetnik pri rješavanju zadataka. Ne treba ih učiti napamet. U lokalnim pregledima navedeni su novouvedeni zapisi, a ovdje su objedinjeni i ranije poznati alati korišteni u ovoj skripti. Nazivi vlastitih funkcija ovise o zadatku; u tablici sintakse služe kao konkretni primjeri.

12.1 Definiranje, pozivanje i uvoz

Pregled sintakse

Zapis	Namjena
<code>def</code>	Ključna riječ za definiranje funkcije
<code>return</code>	Završava poziv funkcije i vraća zadanu vrijednost, odnosno <code>None</code> ako vrijednost nije navedena
<code>global naziv</code>	Omogućuje dodjelu globalnoj varijabli unutar funkcije
<code>is None</code>	Provjerava je li vrijednost upravo <code>None</code>
<code>import math</code>	Uvozi modul pod njegovim nazivom
<code>from math import sqrt</code>	Uvozi odabranu funkciju
<code>import statistics as stat</code>	Uvozi modul pod aliasom
<code>"""Opis funkcije."""</code>	Dokumentacijski tekst kada se nalazi na početku tijela
<code>assert uvjet</code>	Provjerava tvrdnju tijekom isprobavanja programa

12.2 Ugrađene funkcije

Pregled ugrađenih funkcija

Funkcija	Namjena u ovoj vježbi
<code>dir(objekt)</code>	Vraća popis naziva dostupnih u objektu
<code>enumerate(zbirka)</code>	Prolazi kroz redne indekse i pripadajuće vrijednosti
<code>help(objekt)</code>	Prikazuje dokumentaciju zadanog objekta

Funkcija	Namjena u ovoj vježbi
<code>len(zbirka)</code>	Vraća broj elemenata
<code>max(zbirka)</code>	Vraća najveći element neprazne zbirke
<code>min(zbirka)</code>	Vraća najmanji element neprazne zbirke
<code>print(...)</code>	Ispisuje podatke u terminal
<code>round(broj, decimale)</code>	Zaokružuje broj
<code>sorted(zbirka)</code>	Vraća novu sortiranu listu
<code>sum(zbirka)</code>	Zbraja brojčane elemente

12.3 Funkcije modula i metode

🚩 Pregled gotovih alata	
Zapis	Namjena
<code>math.sqrt(broj)</code>	Kvadratni korijen nenegativnog broja
<code>math.pi</code>	Konstanta π , bez poziva zagradama
<code>statistics.mean(vrijednosti)</code>	Aritmetička sredina nepraznih podataka
<code>statistics.median(vrijednosti)</code>	Medijan nepraznih podataka
<code>lista.append(vrijednost)</code>	Dodaje element u postojeću listu
<code>tekst.strip()</code>	Vraća tekst bez rubnih razmaka
<code>tekst.upper()</code>	Vraća tekst s velikim slovima
<code>tekst.title()</code>	Vraća tekst s velikim početnim slovima riječi i ostalim malim slovima

Za dodatna objašnjenja dostupne su službene upute za definiranje funkcija i rad s modulima. Primjeri i zadaci u nastavku koriste gradivo objašnjeno u ovoj skripti.

13 Samostalni zadaci

Svaki zadatak najprije pokušajte riješiti samostalno. Funkciju definirajte iznad poziva i provjerite navedene primjere. Kada zadatak traži vraćanje vrijednosti, ispis napravite izvan funkcije. Rješenja u posljednjoj cjelini služe za usporedbu postupka i rezultata nakon vlastitog pokušaja.

13.1 Zadatak 1

Napišite funkciju `ispisi_naslov` koja prima naziv istraživanja i ispisuje ga između dviju linija od pet znakova `=`. Pozovite je za `Kvaliteta zraka` i `Potrošnja vode`. Ispod svakog naslova, izvan funkcije, ispišite poruku `Početak obrade`. Funkcija u ovom zadatku služi samo ispisu.

13.2 Zadatak 2

Napišite funkciju `vrijednost_zalihe` koja prima cijenu jednog proizvoda i količinu na zalihi te vraća njihovu ukupnu vrijednost. Pozovite funkciju za proizvod cijene `7.50` EUR i količinu `8` bez imenovanih argumenata, a zatim sa imenovanim argumentima za cijenu `12` EUR i količinu `3`. Spremite rezultat u varijablu i ispišite ga na dvije decimale.

13.3 Zadatak 3

Napišite funkciju `udio_u_postocima` koja prima broj prihvaćenih zapisa i ukupan broj zapisa. Ako je ukupan broj `0`, funkcija treba vratiti `None`, inače vraća postotak prihvaćenih zapisa. Pretpostavite da su oba argumenta nenegativni cijeli brojevi i da broj prihvaćenih nije veći od ukupnog. Provjerite slučajeve `18` od `24`, `0` od `10` i `0` od `0`. Izvan funkcije za izostanak rezultata ispišite `Nema zapisa`, a za brojčani rezultat postotak na jednu decimalu.

13.4 Zadatak 4

Napišite funkciju `opis_temperature` koja prima temperaturu. Za vrijednosti manje od `10` vraća `Hladno`, od `10` do `25` vraća `Umjereno`, a od `25` nadalje vraća `Toplo`. Za podatke `[9, 10, 24, 25]` ispišite svaku temperaturu i pripadajući opis.

13.5 Zadatak 5

Napišite funkciju `broj_valjanih` koja prima listu bodova i vraća broj elemenata u rasponu od `0` do `100`, uključujući obje granice. Za listu `[-3, 0, 58, 100, 104]` izračunajte rezultat. Provjerite i praznu listu. Funkcija ne mijenja primljene podatke.

13.6 Zadatak 6

Napišite funkciju `cijena_s_popustom` koja prima početnu cijenu i postotak popusta. Uobičajeni popust iznosi `10` posto, ali treba ga biti moguće promijeniti pri pozivu. Funkcija vraća konačnu cijenu jednog proizvoda. Pretpostavite nenegativnu cijenu i popust od `0` do `100`. Za cijenu `80` EUR provjerite uobičajeni popust, popust od `25` posto i kupnju bez popusta. Ispišite sve tri cijene na dvije decimale.

13.7 Zadatak 7

U glavnom programu postavite `prag = 60`. Napišite funkciju `povecaj_prag` koja prima prag i vraća vrijednost veću za `5`. Spremite povratnu vrijednost u zasebnu varijablu i ispišite početni i novi prag. Zatim promijenite poziv tako da se vraćeni rezultat spremi u početnu varijablu `prag` i ponovno je ispišite.

13.8 Zadatak 8

Napišite funkciju `cijene_s_naknadom` koja prima listu cijena i fiksnu naknadu po proizvodu te vraća novu listu uvećanih cijena. Za listu `[10, 25, 40]` i naknadu `2` ispišite izvornu i novu listu. Izvorni podaci trebaju ostati sačuvani. Provjerite i prazan ulaz.

13.9 Zadatak 9

Napišite funkciju `pregled_bodova` koja prima listu bodova i vraća rječnik s ključevima `broj` i `prolazni`. Prvi sadrži ukupan broj rezultata, a drugi broj rezultata s najmanje `60` bodova. Pretpostavite da su svi ulazni bodovi valjani. Za listu `[55, 60, 82, 40, 91]` ispišite oba podatka dohvaćanjem prema ključu. Prazna lista treba dati dvije nule.

13.10 Zadatak 10

Napišite funkciju `prva_duga_oznaka` koja prima listu tekstualnih oznaka i vraća prvu oznaku dulju od `4` znaka. Ako takve oznake nema, vraća `None`. Provjerite liste `["A1", "B2034", "C5678"]`, `["A1", "B2"]` i praznu listu. Funkcija treba sačuvati redoslijed traženja i vratiti samo prvi odgovarajući rezultat.

13.11 Zadatak 11

Zadana su trajanja postupka u sekundama: `[12, 14, 13, 15, 46]`. Uvezite cijeli modul `statistics`, izračunajte prosjek i medijan trajanja te oba rezultata ispišite s jasnim oznakama.

13.12 Zadatak 12

Površina kvadrata iznosi `144` kvadratnih centimetara. Iz modula `math` uvezite funkciju koja vam je potrebna za izračun duljine stranice. Izračunajte duljinu stranice i ispišite rezultat s jasnom oznakom.

Za kvadrat vrijedi $P = a^2$, pa je duljina stranice $a = \sqrt{P}$, pri čemu je P površina kvadrata.

13.13 Zadatak 13

U modulu `obrada_teksta.py` napišite funkciju `uredi_grad` koja uklanja rubne razmake i pretvara slova u velika. Funkcija prima tekst i vraća uređeni tekst. U modulu ostavite samo definiciju funkcije. U drugoj datoteci, `gradovi.py`, uvezite modul i njegovom funkcijom uredite oznake `" rovinj "` i `"PAZIN "`. Pri pokretanju `gradovi.py` trebaju se ispisati samo ta dva grada.

14 Rješenja samostalnih zadataka

Prikazana su moguća rješenja. Drukčiji nazivi pomoćnih varijabli ili drukčije grananje mogu biti jednako ispravni ako program zadovoljava sve uvjete zadatka. Uz ispis provjerite i vraća li funkcija traženi tip te čuva li ulazne podatke kada je to zatraženo.

14.1 Zadatak 1

▼ Prikaži rješenje i ispis

```
def ispisi_naslov(naziv):  
    print("="*5, naziv, "="*5)  
  
ispisi_naslov("Kvaliteta zraka")  
print("Početak obrade")  
ispisi_naslov("Potrošnja vode")  
print("Početak obrade")
```

```
===== Kvaliteta zraka =====  
Početak obrade  
===== Potrošnja vode =====  
Početak obrade
```

14.2 Zadatak 2

▼ Prikaži rješenje i ispis

```
def vrijednost_zalihe(cijena, kolicina):  
    return cijena * kolicina  
  
vrijednost = vrijednost_zalihe(7.50, 8)  
print(f"Prvi proizvod: {vrijednost:.2f} EUR")  
druga = vrijednost_zalihe(kolicina=3, cijena=12)  
print(f"Drugi proizvod: {druga:.2f} EUR")
```

```
Prvi proizvod: 60.00 EUR  
Drugi proizvod: 36.00 EUR
```

14.3 Zadatak 3

▼ Prikaži rješenje i ispis

```
def udio_u_postocima(prihvaceni, ukupno):  
    if ukupno == 0:  
        return None  
    return prihvaceni / ukupno * 100  
  
for prihvaceni, ukupno in [(18, 24), (0, 10), (0, 0)]:  
    rezultat = udio_u_postocima(prihvaceni, ukupno)  
    if rezultat is None:  
        print("Nema zapisa")  
    else:  
        print(f"{rezultat:.1f} %")
```

75.0 %

0.0 %

Nema zapisa

Nula prihvaćenih od deset zapisa daje valjan postotak nula. Izostanak svih zapisa daje `None` i zato se provjerava izričito, umjesto običnog uvjeta `if rezultat`.

14.4 Zadatak 4

▼ Prikaži rješenje i ispis

```
def opis_temperature(temperatura):  
    if temperatura < 10:  
        return "Hladno"  
    elif temperatura < 25:  
        return "Umjereno"  
    else:  
        return "Toplo"  
  
for temperatura in [9, 10, 24, 25]:  
    print(temperatura, opis_temperature(temperatura))
```

9 Hladno

10 Umjereno

24 Umjereno

25 Toplo

14.5 Zadatak 5

▼ Prikaži rješenje i ispis

```
def broj_valjanih(bodovi):  
    broj = 0  
    for rezultat in bodovi:  
        if 0 <= rezultat <= 100:  
            broj += 1  
    return broj  
  
print(broj_valjanih([-3, 0, 58, 100, 104]))  
print(broj_valjanih([]))
```

```
3  
0
```

14.6 Zadatak 6

▼ Prikaži rješenje i ispis

```
def cijena_s_popustom(cijena, popust=10):  
    return cijena * (1 - popust / 100)  
  
print(f"{cijena_s_popustom(80):.2f} EUR")  
print(f"{cijena_s_popustom(80, popust=25):.2f} EUR")  
print(f"{cijena_s_popustom(80, popust=0):.2f} EUR")
```

```
72.00 EUR  
60.00 EUR  
80.00 EUR
```

14.7 Zadatak 7

▼ Prikaži rješenje i ispis

```
def povecaj_prag(prag):  
    prag = prag + 5  
    return prag  
  
prag = 60  
novi_prag = povecaj_prag(prag)  
print("Početni prag:", prag)  
print("Novi prag:", novi_prag)  
  
prag = povecaj_prag(prag)  
print("Nakon dodjele:", prag)
```

Početni prag: 60
Novi prag: 65
Nakon dodjele: 65

14.8 Zadatak 8

▼ Prikaži rješenje i ispis

```
def cijene_s_naknadom(cijene, naknada):  
    nove_cijene = []  
    for cijena in cijene:  
        nove_cijene.append(cijena + naknada)  
    return nove_cijene  
  
izvorne = [10, 25, 40]  
nove = cijene_s_naknadom(izvorne, 2)  
  
print("Izvorne:", izvorne)  
print("Nove:", nove)  
print("Prazan ulaz:", cijene_s_naknadom([], 2))
```

Izvorne: [10, 25, 40]
Nove: [12, 27, 42]
Prazan ulaz: []

14.9 Zadatak 9

▼ Prikaži rješenje i ispis

```
def pregled_bodova(bodovi):  
    prolazni = 0  
    for rezultat in bodovi:  
        if rezultat >= 60:  
            prolazni += 1  
    return {"broj": len(bodovi), "prolazni": prolazni}
```

```
pregled = pregled_bodova([55, 60, 82, 40, 91])  
print("Broj rezultata:", pregled["broj"])  
print("Prolazni:", pregled["prolazni"])  
print("Prazna lista:", pregled_bodova([]))
```

Broj rezultata: 5

Prolazni: 3

Prazna lista: {'broj': 0, 'prolazni': 0}

14.10 Zadatak 10

▼ Prikaži rješenje i ispis

```
def prva_duga_oznaka(oznake):  
    for oznaka in oznake:  
        if len(oznaka) > 4:  
            return oznaka  
    return None  
  
print(prva_duga_oznaka(["A1", "B2034", "C5678"]))  
print(prva_duga_oznaka(["A1", "B2"]))  
print(prva_duga_oznaka([]))
```

B2034

None

None

14.11 Zadatak 11

▼ Prikaži rješenje i ispis

```
import statistics

trajanja = [12, 14, 13, 15, 46]

print("Prosjek:", statistics.mean(trajanja))
print("Medijan:", statistics.median(trajanja))
```

Prosjek: 20
Medijan: 14

14.12 Zadatak 12

▼ Prikaži rješenje i ispis

```
from math import sqrt

print("Stranica kvadrata:", sqrt(144))
```

Stranica kvadrata: 12.0

14.13 Zadatak 13

▼ Prikaži rješenje i ispis

Datoteka `obrada_teksta.py`:

```
def uredi_grad(grad):
    return grad.strip().upper()
```

Datoteka `gradovi.py`:

```
import obrada_teksta

print(obrada_teksta.uredi_grad(" rovinj "))
print(obrada_teksta.uredi_grad("PAZIN "))
```

Ispis pri pokretanju `gradovi.py`:

ROVINJ
PAZIN